Rapport de TP – Word2Vec avec Gensim

Résumés de films et avis clients Amazon

Salma Alaoui Mrani

15 octobre 2026

Table des matières

1 Introduction **2**

Partie I – Le modèle Word2Vec **2**

2 Représenter un mot par des nombres **2**

3 Word2Vec et l'architecture Skip-gram **2**

4 La similarité cosinus **3**

Partie II – Mise en pratique et résultats **3**

5 Les deux jeux de données **3**

6 Préparation des textes **3**

7 Paramètres d'entraînement **4**

8 Résultats sur les résumés de films **4**

9 Résultats sur les avis Amazon **5**

10 Similarité entre deux mots **5**

11 Test d'analogies **6**

12 Visualisation des mots **6**

13 Comparaison des deux expériences **8**

14 Limites **8**

15 Conclusion **8**

1 Introduction

L'objectif de ce TP est d'entraîner le modèle Word2Vec avec la bibliothèque Python Gensim sur deux jeux de données :

— des **résumés de films** (*The Movies Dataset*, Kaggle) ;

— des **avis clients Amazon** sur des téléphones et accessoires (Stanford SNAP).

La première partie explique le fonctionnement de Word2Vec. La seconde présente la mise en pratique et analyse les résultats obtenus sur chaque corpus.

Partie I – Le modèle Word2Vec

2 Représenter un mot par des nombres

Un ordinateur ne comprend pas directement les mots. Pour les traiter, on doit les transformer en nombres. Un **word embedding** (plongement lexical) associe à chaque mot une liste de nombres, appelée **vecteur**. Dans ce TP, chaque mot est représenté par 100 nombres.

L'idée de base est simple : **on reconnaît un mot aux mots qui l'entourent**. Par exemple, *good* et *great* apparaissent dans des phrases très semblables (« the battery is good », « the battery is great »). Le modèle va donc leur donner des vecteurs proches.

Attention : deux mots proches pour le modèle ne sont pas forcément synonymes. Ils sont simplement utilisés dans des contextes similaires.

3 Word2Vec et l'architecture Skip-gram

Word2Vec a été proposé par Mikolov et al. en 2013 [4]. C'est un petit réseau de neurones qui apprend les vecteurs des mots à partir d'un grand nombre de textes. Il existe deux versions :

— **CBOW** : le modèle devine un mot à partir des mots qui l'entourent ;

— **Skip-gram** : le modèle fait l'inverse, il devine les mots qui entourent un mot donné.

Nous avons utilisé **Skip-gram** (paramètre `sg=1` dans Gensim).

Exemple. Prenons la phrase « *…after the war ended in …* ». Pour le mot central *war*, avec une fenêtre de 2 mots de chaque côté, le modèle essaie de deviner *after*, *the*, *ended* et *in* (figure 1). Dans notre TP, la fenêtre est de 5 mots de chaque côté.

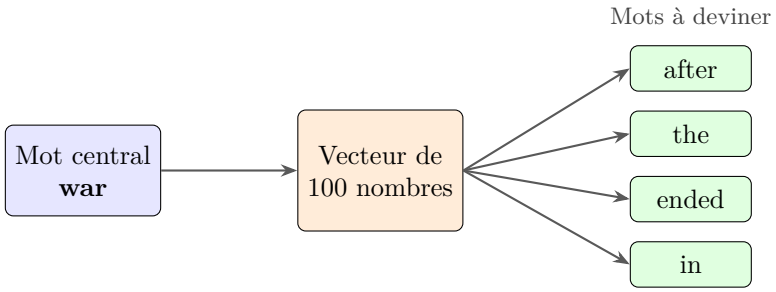


Figure 1 – Principe de Skip-gram : à partir d'un mot, le modèle apprend à deviner ses voisins dans la phrase. Le vecteur appris au milieu est le « word embedding » du mot.

Au début, les vecteurs sont choisis au hasard. Le modèle parcourt ensuite tout le corpus et corrige progressivement les vecteurs pour mieux deviner les mots voisins. À la fin, ce n'est pas la

prédiction qui nous intéresse, mais les **vecteurs appris** : deux mots qui ont souvent les mêmes voisins finissent avec des vecteurs proches.

4 La similarité cosinus

Pour mesurer si deux mots sont proches, on compare leurs vecteurs avec la **similarité cosinus** :

$$\cos(u, v) = \frac{u \cdot v}{\|u\| \times \|v\|}$$

On peut imaginer chaque vecteur comme une flèche. Le cosinus mesure si deux flèches pointent dans la même direction :

— proche de **1** : les mots apparaissent dans des contextes très semblables ;

— proche de **0** : les mots ont peu de contextes en commun.

La fonction `most_similar` de Gensim donne les mots qui ont le cosinus le plus élevé avec un mot choisi. Un cosinus élevé indique que deux mots sont **utilisés de la même façon** dans le corpus, mais il ne prouve pas qu'ils ont le même sens.

Partie II – Mise en pratique et résultats

5 Les deux jeux de données

Corpus A – résumés de films. Nous avons utilisé le fichier `movies_metadata.csv` de *The Movies Dataset* [1], et plus précisément la colonne `overview`, qui contient un court résumé de chaque film. Ces textes sont bien rédigés et parlent de sujets très variés.

Corpus B – avis Amazon. Nous avons utilisé le fichier `reviews.json.gz` [2], qui contient des avis sur des téléphones et accessoires, et plus précisément le champ `reviewText`. Ces textes sont écrits par des clients : ils contiennent des fautes d'orthographe, des abréviations et des mots collés.

6 Préparation des textes

Avant l'entraînement, chaque texte est découpé en une liste de mots (« tokens »). Nous avons appliqué les étapes suivantes aux deux corpus :

— suppression des textes vides ;

— passage en minuscules ;

— découpage en mots en ne gardant que les lettres : la ponctuation, les chiffres et les accents sont supprimés ;

— suppression des mots d'une seule lettre, sauf « i ».

Pour les avis Amazon, nous avons aussi corrigé quelques fautes fréquentes (par exemple *quaility* → *quality*, *phoen* → *phone*) et séparé quelques mots collés (par exemple *phoneit* → *phone it*). Ces corrections ne concernent qu'une courte liste de cas repérés à la main : beaucoup d'autres fautes restent dans le corpus.

7 Paramètres d'entraînement

Les deux modèles ont été entraînés avec les mêmes réglages, ce qui permet de comparer leurs résultats.

Table 1 – Paramètres utilisés (Gensim 4.4.0).

Paramètre	Valeur	Signification
`sg`	1	Architecture Skip-gram
`vector_size`	100	Chaque mot est représenté par 100 nombres
`window`	5	Le modèle regarde 5 mots de chaque côté
`min_count`	5	Les mots vus moins de 5 fois sont ignorés
`epochs`	10	Le modèle parcourt le corpus 10 fois

8 Résultats sur les résumés de films

Table 2 – Corpus A : chiffres clés.

Résumés au départ	45 466
Résumés gardés après préparation	44 493
Nombre total de mots (tokens)	2 362 992
Vocabulaire (mots différents gardés)	22 573
Temps d'entraînement	31,63 s

Table 3 – Corpus A : les 5 mots les plus proches (similarité cosinus).

Rang	`love`	`war`	`family`	`death`	`man`
1	madly (0,71)	ii (0,83)	families (0,60)	deaths (0,69)	woman (0,68)
2	romance (0,60)	civil (0,69)	mikkel (0,59)	disappearance (0,63)	person (0,59)
3	fantasize (0,60)	wwii (0,67)	parents (0,58)	demise (0,60)	fireman (0,57)
4	persists (0,58)	ww (0,67)	community (0,58)	grieving (0,60)	men (0,57)
5	friendship (0,58)	vietnam (0,67)	drifting (0,56)	carnage (0,59)	pediatrician (0,57)

Ce que nous observons.

- **`death`** donne les résultats les plus cohérents : tous ses voisins parlent de la mort ou de la perte (*deaths*, *demise*, *grieving*).
- **`war`** est entouré de guerres réelles (*civil*, *wwii*, *vietnam*). Le mot *ii* arrive en premier car l'expression *World War II* est très fréquente dans les résumés. Comme les chiffres sont supprimés, *WW 2* devient aussi *ww*.
- **`love`** : *romance* et *friendship* sont logiques. *Madly* arrive en premier à cause de l'expression *madly in love* : le modèle a appris que ces mots vont souvent ensemble, pas qu'ils ont le même sens.
- **`man`** : le mot le plus proche est *woman*. Les deux mots ne sont pas synonymes, mais ils sont utilisés de la même façon (« a young man / woman who… »).

— **`family`** : *families* et *parents* sont logiques. En revanche, *mikkel* est un prénom qui n'apparaît que 5 fois, dans des films sur la famille. Le modèle a très peu d'exemples pour ce mot, donc son vecteur est peu fiable.

9 Résultats sur les avis Amazon

Table 4 – Corpus B : chiffres clés.

Avis gardés	194 329
Avis vides ou inutilisables retirés	110
Nombre total de mots (tokens)	17 326 224
Vocabulaire (mots différents gardés)	20 853
Temps d'entraînement	164,04 s

Table 5 – Corpus B : les 5 mots les plus proches (similarité cosinus).

Rang	`phone`	`battery`	`screen`	`quality`	`good`
1	cellphone (0,69)	batter (0,88)	glass (0,78)	build (0,73)	decent (0,84)
2	iphone (0,64)	batt (0,75)	film (0,73)	workmanship (0,71)	great (0,78)
3	device (0,64)	capacity (0,75)	screens (0,72)	craftsmanship (0,68)	fantastic (0,71)
4	lgg (0,61)	mah (0,74)	protector (0,72)	clarity (0,65)	excellent (0,70)
5	smartphone (0,61)	batteries (0,72)	screensaver (0,68)	qualitycons (0,65)	nice (0,70)

Ce que nous observons.

— **`phone`** : *cellphone*, *smartphone*, *iphone* et *device* sont très logiques. *Lgg* vient du modèle « LG G2 » écrit « LGG2 » : le chiffre a été supprimé.

— **`battery`** : les deux premiers voisins sont *batter* (une faute de frappe, comme dans « batter life ») et *batt* (une abréviation). Le modèle a donc reconnu tout seul ces variantes, mais cela montre aussi que notre nettoyage ne les a pas corrigées. *Mah* vient de l'unité de capacité « mAh » (par exemple « 3000mAh »).

— **`screen`** : les voisins (*glass*, *film*, *protector*) parlent surtout de protections d'écran, un produit très vendu dans cette catégorie.

— **`quality`** : *build* vient de l'expression *build quality*. *Qualitycons* est une erreur : il vient d'avis écrits « good qualityCons : . . . » sans espace.

— **`good`** : tous les voisins sont des adjectifs positifs. Le modèle ne fait pas la différence d'intensité : *decent* est plus proche de *good* que *excellent*.

10 Similarité entre deux mots

Table 6 – Similarité cosinus des deux paires testées.

Corpus	**Paire**	**Cosinus**
Films	`love` / `family`	0,2201
Avis Amazon	`battery` / `charging`	0,5218

— **`love / family` (0,22)** : la valeur est faible. Dans les résumés, *love* est surtout utilisé pour parler de romance, et *family* pour parler de parents et de proches. Les deux mots apparaissent donc rarement dans les mêmes contextes.

— **`battery / charging` (0,52)** : la valeur est moyenne. Les deux mots sont liés (on charge une batterie) et apparaissent souvent dans les mêmes phrases, mais ils ne désignent pas la même chose.

Ces deux chiffres ne permettent pas de dire qu'un modèle est meilleur que l'autre : ils concernent des mots différents, dans deux modèles entraînés séparément.

11 Test d'analogies

Pour obtenir une mesure chiffrée, nous avons utilisé le test d'analogies de Mikolov et al. [4], fourni avec Gensim. Chaque question a la forme : « *Athens* est à *Greece* ce que *Berlin* est à . . . ? ». Le modèle calcule *greece* − *athens* + *berlin* et propose le mot dont le vecteur est le plus proche du résultat. La réponse est juste s'il trouve *germany*.

Le test contient 19 544 questions. Une question est ignorée si l'un de ses mots n'est pas dans le vocabulaire du modèle.

Table 7 – Résultats du test d'analogies.

Indicateur	**Films**	**Avis Amazon**
Questions testées (sur 19 544)	8 703	7 392
Bonnes réponses	824	2 187
Score global	**9,5 %**	**29,6 %**
Catégorie « famille » (*boy / girl*, *brother / sister*. . .)	51,0 %	40,9 %
Catégorie « comparatifs » (*good / better*. . .)	6,6 %	67,9 %

Ce que nous observons.

— Les scores sont globalement faibles. Ce test a été conçu pour des modèles entraînés sur des corpus beaucoup plus grands et plus généraux. De plus, une grande partie des questions porte sur la géographie (capitales, villes) et les monnaies, des sujets rares dans nos textes.

— Les deux modèles réussissent bien la catégorie « famille ». Les relations entre personnes sont fréquentes dans les résumés de films comme dans les avis.

— Le modèle des avis réussit très bien les comparatifs (67,9 %). Dans leurs avis, les clients comparent souvent les produits entre eux.

— Les deux modèles ne sont pas testés sur les mêmes questions, car leurs vocabulaires sont différents. Un score plus élevé ne prouve donc pas qu'un modèle est meilleur en général.

12 Visualisation des mots

Pour visualiser les résultats, nous avons réduit les vecteurs de 100 nombres à 2 nombres avec une analyse en composantes principales (ACP). Cela permet de placer chaque mot sur un graphique. Cette réduction perd une partie de l'information : les figures montrent des groupes, mais les distances ne sont qu'approximatives.

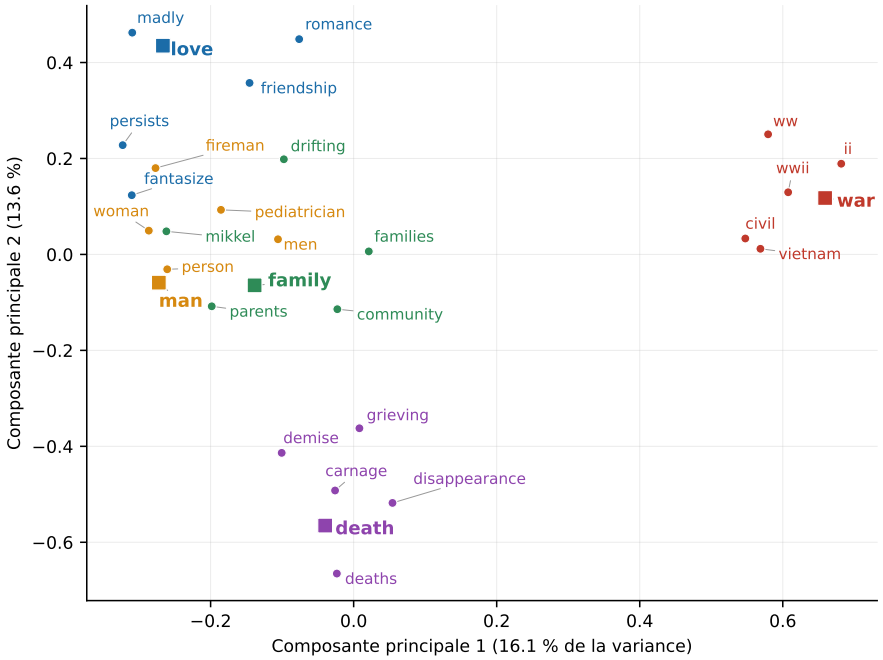


Figure 2 – Résumés de films : mots étudiés (carrés) et leurs 5 plus proches voisins (points).

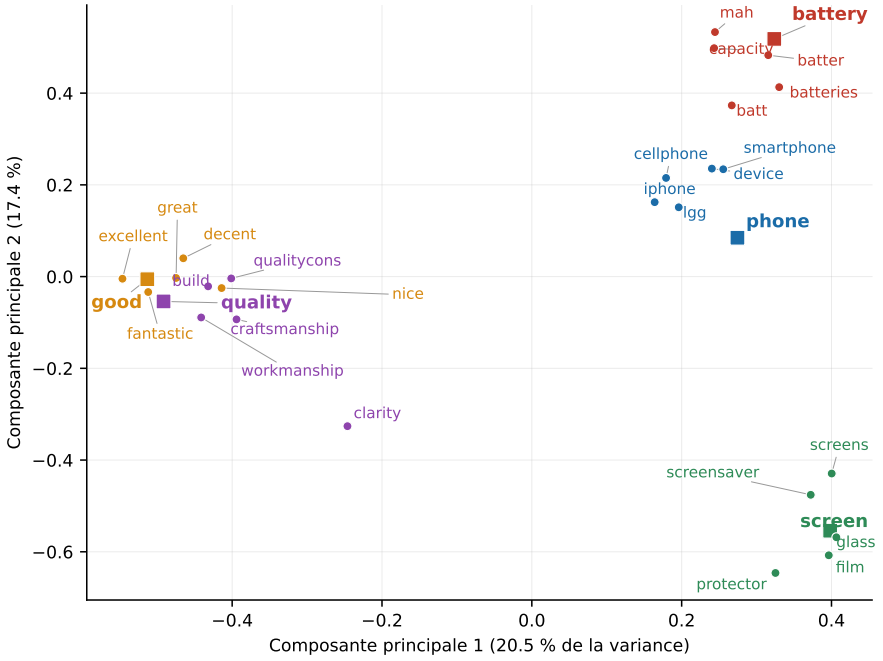


Figure 3 – Avis Amazon : mots étudiés (carrés) et leurs 5 plus proches voisins (points).

Sur la figure 2, les groupes `war` et `death` sont bien séparés des autres. Sur la figure 3, on distingue nettement trois groupes : la batterie, le téléphone et l'écran. Les mots `good` et `quality` sont proches, car ils servent tous les deux à donner un avis (« good quality »).

Table 8 – Comparaison des deux corpus.

Indicateur	Films	Avis Amazon
Nombre de mots (tokens)	2 362 992	17 326 224
Vocabulaire	22 573	20 853
Temps d'entraînement	31,63 s	164,04 s

13 Comparaison des deux expériences

Temps d'entraînement. Le corpus d'avis contient environ 7 fois plus de mots et son entraînement a pris environ 5 fois plus de temps. La taille du corpus joue donc un rôle important, mais elle n'est pas le seul facteur : le temps dépend aussi de l'ordinateur utilisé.

Taille du vocabulaire. Le corpus d'avis a beaucoup plus de mots au total, mais un vocabulaire un peu plus petit. Les résumés de films parlent de sujets très variés (personnages, lieux, époques) et utilisent donc beaucoup de mots différents. Les avis Amazon parlent presque toujours des mêmes produits et répètent les mêmes mots. Le nombre de mots différents dépend donc de la variété des textes et du seuil `min_count`, pas seulement de la taille du corpus.

Type de résultats. Le modèle des films trouve surtout des mots liés par le thème (guerre, mort, famille). Le modèle des avis trouve surtout des mots techniques et des avis positifs, mais aussi plus de fautes d'orthographe. Chaque modèle reflète donc le type de texte sur lequel il a été entraîné.

14 Limites

— **Une évaluation limitée** : nous avons étudié 10 mots, 2 paires et un seul test chiffré. Ce test porte surtout sur des sujets (géographie, monnaies) peu présents dans nos textes.

— **Un nettoyage incomplet** : des fautes et des mots collés restent dans les avis (*batter*, *batt*, *qualitycons*).

— **La suppression des chiffres** fait perdre de l'information (*World War II* donne *ii*, *LGG2* donne *lgg*).

— **Un seul vecteur par mot** : un mot qui a plusieurs sens (*film*, *screensaver*) mélange tous ses sens dans un seul vecteur.

— **Les mots rares** ont des vecteurs peu fiables, comme *mikkel*, vu seulement 5 fois.

15 Conclusion

Dans ce TP, nous avons entraîné deux modèles Word2Vec (Skip-gram) avec Gensim : l'un sur des résumés de films, l'autre sur des avis clients Amazon.

Les deux modèles donnent des résultats globalement cohérents. Le modèle des films regroupe des mots d'un même thème, comme *death* et *grieving*. Le modèle des avis regroupe des mots liés aux produits, comme *battery* et *capacity*.

Ce travail montre aussi deux points importants. D'abord, deux mots proches pour le modèle ne sont pas forcément synonymes : ils sont simplement utilisés de la même façon (*man* et *woman*). Ensuite, la qualité des résultats dépend beaucoup de la préparation des textes : les fautes non corrigées, comme *batter*, apparaissent directement dans les résultats.

Le test d'analogies nuance ces observations. Le modèle des avis obtient 29,6 % de bonnes réponses contre 9,5 % pour celui des films, mais les deux modèles ne répondent pas aux mêmes questions. Le score des avis repose surtout sur les comparatifs (67,9 %), très fréquents dans les avis, alors que le modèle des films fait mieux sur la catégorie « famille » (51,0 % contre 40,9 %). Chaque modèle réussit donc les relations dont parle son corpus.

Références

[1] R. Banik. *The Movies Dataset*. Kaggle. https://www.kaggle.com/rounakbanik/the-movies-dataset

[2] J. McAuley. *Amazon product data – Cell Phones and Accessories*. Stanford SNAP. http://snap.stanford.edu/data/amazon/productGraph/categoryFiles/reviews_Cell_Phones_and_Accessories_5.json.gz

[3] Documentation Gensim – Word2Vec. https://radimrehurek.com/gensim/models/word2vec.html

[4] T. Mikolov, K. Chen, G. Corrado et J. Dean (2013). *Efficient Estimation of Word Representations in Vector Space*. arXiv :1301.3781.

[5] Ressource du TP : tutoriel vidéo Word2Vec avec Gensim. https://youtu.be/Q2NtCcqmIww

[6] Ressource du TP : notebook *TP4 – Deep learning et NLP*. https://sally14.github.io/teaching/notebooks/TP4_dl_nlp.html